

Project Development Documentation

Sentiment Analysis for Social Media Data using MongoDB and MapReduce

Below is a complete roadmap you can follow **from scratch to final project completion**. I recommend building it phase by phase rather than trying to develop everything at once.

1. Project Overview

Project Title

Sentiment Analysis for Social Media Data using MongoDB and MapReduce

Problem Statement

Social media platforms generate a huge volume of textual data every day. Analyzing this data manually to understand public opinion is difficult and inefficient. Organizations, researchers, and businesses need an automated system to process large amounts of social media text and identify whether user opinions are positive, negative, or neutral.

Traditional sequential processing can become inefficient when the volume of data increases. Therefore, this project applies the **MapReduce programming model** to process and aggregate sentiment information from social media posts.

The project uses:

- Social media dataset
 - Python
 - MongoDB
 - Sentiment Analysis
 - MapReduce paradigm
 - Data visualization dashboard
-

2. Main Objective

The main objective of this project is:

To develop a system that collects social media text data, stores it in MongoDB, performs sentiment analysis using Python, and applies the MapReduce model to aggregate and analyze positive, negative, and neutral sentiments.

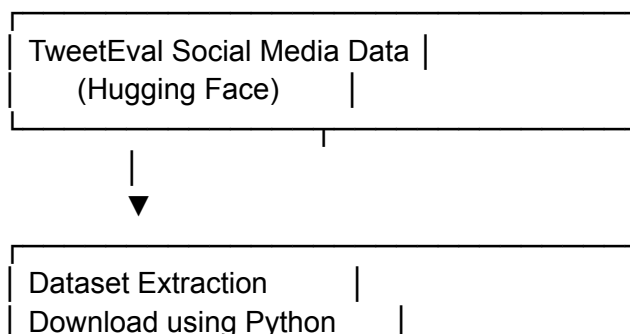
3. Project Objectives

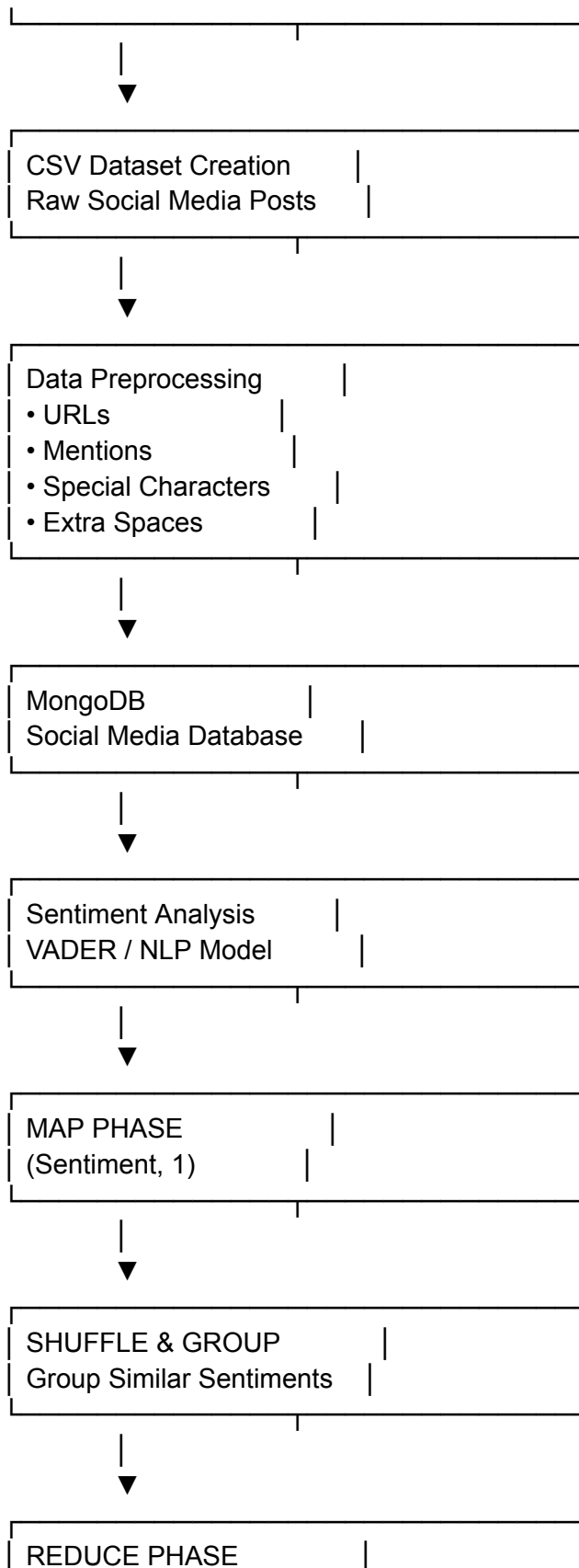
Your project can have the following objectives:

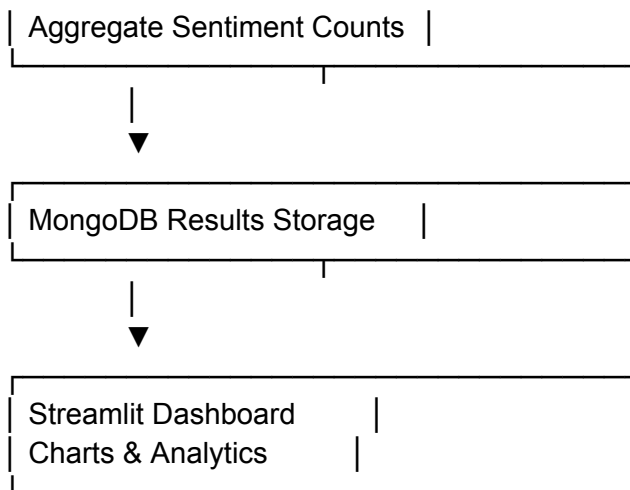
1. Collect a social media sentiment dataset.
 2. Convert and prepare the dataset in CSV format.
 3. Clean and preprocess social media text.
 4. Store large volumes of social media posts in MongoDB.
 5. Perform sentiment analysis on individual posts.
 6. Implement the Map phase to generate sentiment key-value pairs.
 7. Implement Shuffle and Group operations.
 8. Implement the Reduce phase to aggregate sentiment counts.
 9. Store processed results in MongoDB.
 10. Visualize sentiment statistics through an interactive dashboard.
 11. Compare predicted sentiment with actual dataset labels.
-

4. Overall Project Architecture

Your complete project will follow this pipeline:







5. Technology Stack

Layer	Technology
Programming Language	Python
Dataset Source	Hugging Face TweetEval
Dataset Format	CSV
Database	MongoDB
NLP	NLTK
Sentiment Analyzer	VADER
Data Processing	Pandas
MapReduce	Custom Python Implementation
Backend	Python
Frontend Dashboard	Streamlit
Visualization	Plotly
Development Environment	VS Code
Version Control	GitHub

6. Complete Project Phases

I recommend dividing your project into **10 phases**.

PHASE 1 — Project Setup

Goal

Set up the development environment.

Install Python

Make sure Python 3.10+ is installed.

Check:

```
python --version
```

Create Project Folder

sentiment-mapreduce-project/

Recommended structure:

sentiment-mapreduce-project/

```
|
|— data/
|   |— raw/
|   |— processed/
|— src/
|   |— data_collection/
|   |— preprocessing/
|   |— database/
|   |— sentiment_analysis/
|   |— mapreduce/
|   |— evaluation/
|— dashboard/
```

```
|
|— notebooks/
|— results/
|— tests/
|— requirements.txt
|— README.md
|— app.py
```

Create Virtual Environment

```
python -m venv venv
```

Activate:

Windows

```
venv\Scripts\activate
```

Install required libraries:

```
pip install pandas numpy nltk pymongo streamlit plotly datasets scikit-learn python-dotenv
```

Save dependencies:

```
pip freeze > requirements.txt
```

PHASE 2 — Dataset Collection

Goal

Fetch real social media data.

Dataset Source

Use the **TweetEval Sentiment Dataset**.

Official resources:

- [TweetEval dataset on Hugging Face](#)
- [TweetEval official GitHub repository](#)

This dataset is suitable because it contains Twitter/X-style social media posts with three sentiment classes:

0 → Negative

1 → Neutral

2 → Positive

Step 2.1 — Fetch Dataset

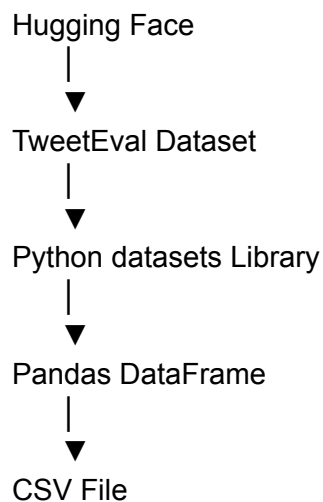
Create:

src/data_collection/download_dataset.py

The program should:

1. Download TweetEval.
2. Select the sentiment dataset.
3. Extract train, validation, and test data.
4. Convert numerical labels to readable labels.
5. Save the data as CSV.

Conceptual flow:



Step 2.2 — Create Raw Dataset

Your CSV should look like:

```
post_id,text,actual_sentiment,split
1,"I love this product!",Positive,train
2,"This is the worst service",Negative,train
3,"The product is okay",Neutral,train
```

Save it as:

```
data/raw/social_media_raw.csv
```

Important Dataset Design Decision

Do not delete the original sentiment label permanently.

Keep:

```
actual_sentiment
```

But do not provide it to the sentiment prediction algorithm.

Your system should predict:

```
predicted_sentiment
```

Later:

```
actual_sentiment
```

VS

```
predicted_sentiment
```

This allows you to evaluate your system.

PHASE 3 — Dataset Exploration

Goal

Understand your data before processing.

Create:

notebooks/dataset_exploration.ipynb

Analyze:

- Number of records
- Number of positive posts
- Number of negative posts
- Number of neutral posts
- Missing values
- Duplicate records
- Text length
- Dataset distribution

Example:

Total Posts: 45,000

Positive: 15,000

Negative: 15,000

Neutral: 15,000

Create initial graphs.

Visualizations

- Sentiment distribution bar chart
- Pie chart
- Post length histogram

This phase helps you explain your dataset during the project presentation.

PHASE 4 — Data Preprocessing

Goal

Clean social media text before sentiment analysis.

Social media posts often contain:

@username

#hashtags
URLs
emojis
RT
special characters
extra spaces

Example raw post:

RT @user OMG!!! 🥰🥰 I absolutely LOVE this product!!!
Check <https://example.com> #Amazing

Cleaned:

omg absolutely love this product amazing

Preprocessing Operations

Your preprocessing pipeline should include:

1. Convert to lowercase

I LOVE THIS

↓

i love this

2. Remove URLs

Check this <https://example.com>

↓

check this

3. Remove mentions

@john I love this

↓

i love this

4. Remove unnecessary symbols

Amazing!!!!!!!

↓

amazing

5. Remove extra spaces

I love this

↓

i love this

Important Note About Emojis

Since VADER is designed for social-media-style text, I recommend **not aggressively removing emojis initially**.

For example:

I love this 🥰

The emoji may contribute to sentiment.

You can experiment with:

- Original text
- Lightly cleaned text

This gives you a good evaluation component.

Output

Save processed data as:

data/processed/social_media_cleaned.csv

Structure:

post_id,text,cleaned_text,actual_sentiment,split

1,"I love this product!","i love this product",Positive,train

PHASE 5 — MongoDB Database Setup

Goal

Store social media data in a NoSQL database.

Install MongoDB

You have two options.

Option A — MongoDB Atlas ★ Recommended

Cloud-based MongoDB.

[MongoDB Atlas](#)

Advantages:

- Free tier available
- No complex local installation
- Accessible online
- Easy to demonstrate

Option B — Local MongoDB

Install MongoDB Community Edition locally.

For your project, I recommend **MongoDB Atlas**.

Database Design

Database name:

sentiment_analysis_db

Collections:

```
sentiment_analysis_db
├── social_posts
├── processing_results
├── mapreduce_results
└── project_metadata
```

Collection 1 — social_posts

Each social media post:

```
{
  "_id": "...",
  "post_id": 1,
  "original_text": "I absolutely love this product!",
  "cleaned_text": "i absolutely love this product",
  "actual_sentiment": "Positive",
  "predicted_sentiment": null,
  "sentiment_score": null,
  "dataset_split": "train",
  "processed": false
}
```

Collection 2 — processing_results

After sentiment analysis:

```
{
  "post_id": 1,
  "predicted_sentiment": "Positive",
  "sentiment_score": 0.85,
  "processed_at": "2026-08-31"
}
```

Collection 3 — mapreduce_results

Final aggregated result:

```
{
  "analysis_id": "run_001",
```

```
"sentiment": "Positive",  
"count": 15420,  
"percentage": 45.2  
}
```

PHASE 6 — Insert Dataset into MongoDB

Goal

Move the cleaned CSV dataset into MongoDB.

Create:

src/database/mongodb_connection.py

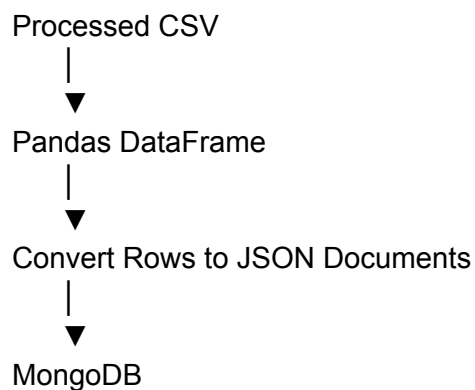
Responsibilities:

- Connect to MongoDB
 - Create database
 - Create collections
 - Insert documents
 - Fetch documents
 - Update documents
-

Create:

src/database/load_data.py

Flow:



↓
social_posts Collection

Batch Insertion

Do not insert thousands of records one by one.

Use batches:

1000 posts
↓
Insert
↓
Next 1000 posts

This is better for performance and demonstrates large-scale data handling.

PHASE 7 — Sentiment Analysis Engine

Goal

Predict the sentiment of each social media post.

Initially use:

VADER Sentiment Analyzer

VADER is useful because it is designed for:

- Social media
- Informal text
- Emojis
- Short sentences

Your sentiment module:

```
src/sentiment_analysis/  
└── sentiment_analyzer.py
```

Processing Logic

Input:

I absolutely love this product!

VADER produces scores:

Positive: 0.75

Neutral: 0.20

Negative: 0.05

Compound: 0.82

Classification:

compound $\geq$ 0.05

↓

Positive

compound $\leq$ -0.05

↓

Negative

otherwise

↓

Neutral

Store Results

After prediction:

MongoDB

Document changes to:

```
{
  "post_id": 1,
  "original_text": "I absolutely love this product!",
  "actual_sentiment": "Positive",
  "predicted_sentiment": "Positive",
  "sentiment_score": 0.82,
```



```
"processed": true  
}
```

PHASE 8 — MapReduce Implementation

★ Core of Project

This is the most important part of your project.

Your MapReduce implementation should have three stages.

Stage 1 — MAP

Each social media post is processed independently.

Example:

Post:

"I love this product"

Sentiment Prediction:

Positive

Mapper Output:

(Positive, 1)

Another:

"This is terrible"

Mapper Output:

(Negative, 1)

After processing:

(Positive, 1)

(Positive, 1)

(Negative, 1)

(Neutral, 1)

(Positive, 1)

Create:

src/mapreduce/mapper.py

Stage 2 — Shuffle and Group

Group similar keys.

Input:

(Positive, 1)

(Positive, 1)

(Negative, 1)

(Neutral, 1)

Output:

Positive → [1, 1]

Negative → [1]

Neutral → [1]

Create:

src/mapreduce/shuffle.py

Stage 3 — Reduce

Add all values.

Input:

Positive → [1, 1, 1, 1]

Negative → [1, 1]

Neutral → [1, 1, 1]

Output:

Positive → 4
Negative → 2
Neutral → 3

Create:

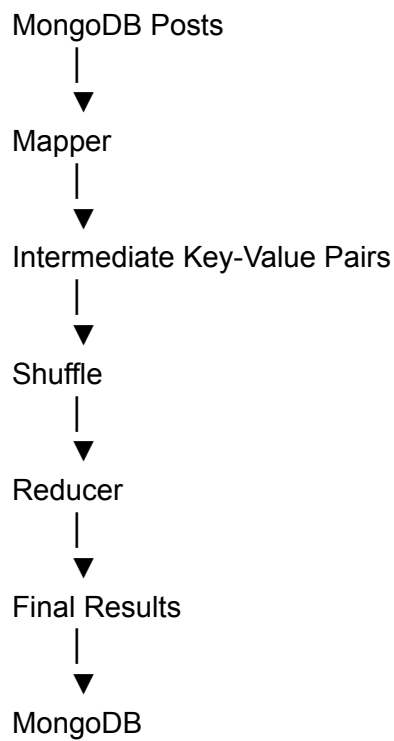
src/mapreduce/reducer.py

MapReduce Controller

Create:

src/mapreduce/mapreduce_engine.py

Responsibilities:



Important Enhancement

Instead of only performing:

Sentiment → Count

You can make your MapReduce implementation more interesting.

For example:

Map by Sentiment

(Positive, 1)

Map by Date

(2026-08-31, Positive)

Map by Topic

(Technology, Positive)

But for Version 1, keep it simple.

PHASE 9 — Evaluation

Goal

Compare predicted sentiment with actual sentiment.

Because TweetEval already contains labels, you can evaluate your sentiment analyzer.

Compare:

Post	Actual	Predicted
I love this	Positive	Positive
This is terrible	Negative	Negative
It is okay	Neutral	Neutral

Calculate:

Accuracy

Correct Predictions

Total Predictions

Precision

Measures how many predicted sentiments were correct.

Recall

Measures how many actual sentiments were correctly identified.

F1 Score

Balances precision and recall.

Use:

scikit-learn

Create:

src/evaluation/evaluate_model.py

PHASE 10 — Dashboard Development

Goal

Create a user-friendly interface.

Use:

Streamlit

Your dashboard:

```
dashboard/  
|  
├── home.py  
├── data_overview.py  
├── sentiment_analysis.py  
├── mapreduce_visualization.py  
└── evaluation.py
```

Or initially create everything in:

app.py

Dashboard Pages

Page 1 — Project Overview

Display:

Sentiment Analysis for Social Media Data
Using MongoDB and MapReduce

Show:

- Project description
 - Total posts
 - Dataset information
-

Page 2 — Dataset Overview

Display:

- Total posts
- Sample posts
- Dataset distribution

Charts:

Positive
Negative
Neutral

Page 3 — Sentiment Analysis

Show:

- Individual social media posts

- Predicted sentiment
- Sentiment score

Example:

Post:

"I love this product!"

Predicted Sentiment:

😊 Positive

Confidence Score:

0.82

Page 4 — MapReduce Visualization ★

This page should visually explain your project.

MAP Output

Positive → 1

Negative → 1

Positive → 1

Neutral → 1

SHUFFLE

Positive → [1,1]

Negative → [1]

Neutral → [1]

REDUCE

Positive → 2

Negative → 1

Neutral → 1

This will make your project presentation much stronger.

Page 5 — Analytics

Show:

Metrics

Total Posts: 50,000

Positive: 20,000

Negative: 15,000

Neutral: 15,000

Charts:

- Pie chart
 - Bar chart
 - Sentiment percentage
-

Page 6 — Model Evaluation

Show:

Accuracy

Precision

Recall

F1 Score

Also include:

Confusion Matrix

Predicted
P N Neu

Actual P

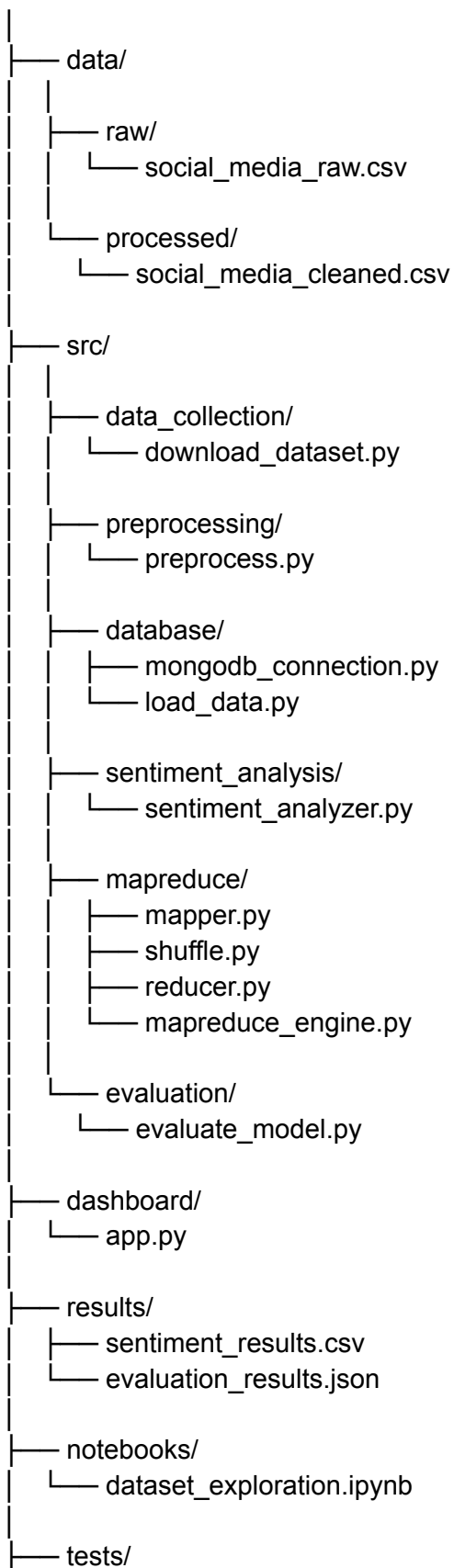
Actual N

Actual Neu

Complete Folder Structure

Your final project should look like this:

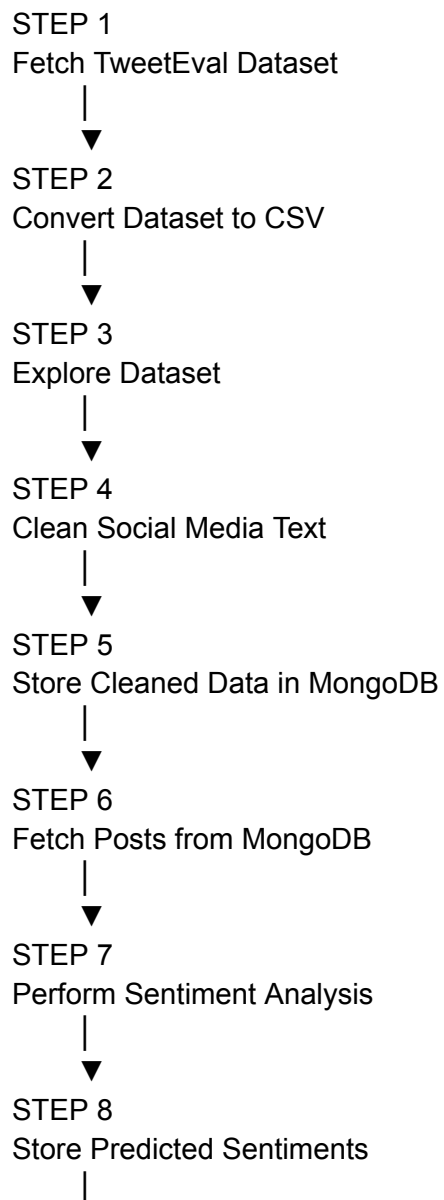
sentiment-analysis-mapreduce/

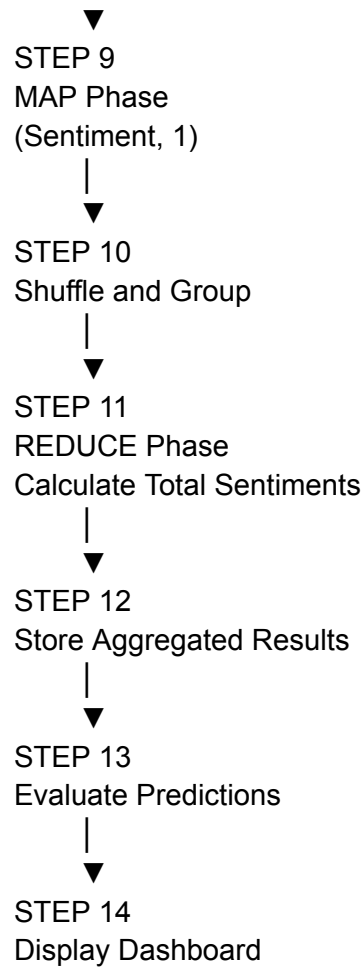


- requirements.txt
- README.md
- .env

Complete Data Flow

This is the final end-to-end workflow of your project:





Recommended Development Order

Do not build everything at once.

Follow this exact order:

Week/Phase 1

- ✓ Create project folder
 - ✓ Setup Python environment
 - ✓ Install libraries
 - ✓ Download TweetEval
 - ✓ Create CSV
-

Week/Phase 2

- ✓ Explore dataset
 - ✓ Clean data
 - ✓ Handle duplicates
 - ✓ Save processed dataset
-

Week/Phase 3

- ✓ Setup MongoDB
 - ✓ Create database
 - ✓ Create collections
 - ✓ Insert dataset
 - ✓ Test retrieval
-

Week/Phase 4

- ✓ Implement sentiment analyzer
 - ✓ Process posts
 - ✓ Store predicted sentiment
-

Week/Phase 5

- ✓ Implement Mapper
 - ✓ Implement Shuffle
 - ✓ Implement Reducer
 - ✓ Store MapReduce results
-

Week/Phase 6

- ✓ Evaluate predictions
 - ✓ Calculate accuracy
 - ✓ Create confusion matrix
-

Week/Phase 7

- ✓ Build Streamlit dashboard
- ✓ Add charts
- ✓ Add MapReduce visualization

- ✓ Connect MongoDB
-

Week/Phase 8

- ✓ Testing
 - ✓ Bug fixing
 - ✓ README documentation
 - ✓ Screenshots
 - ✓ Project presentation
-

Important Point About MongoDB MapReduce

One thing you should be clear about in your documentation:

There are **two possible interpretations of your project title**:

Approach 1

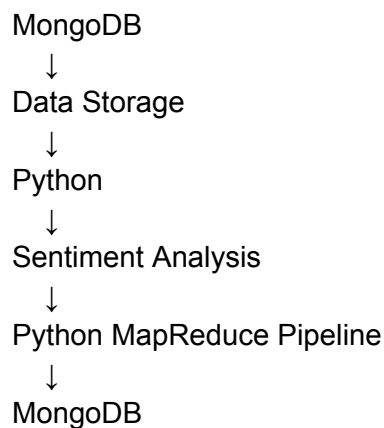
Python implements MapReduce
MongoDB stores the data

Approach 2

MongoDB itself performs MapReduce

For your project, I recommend:

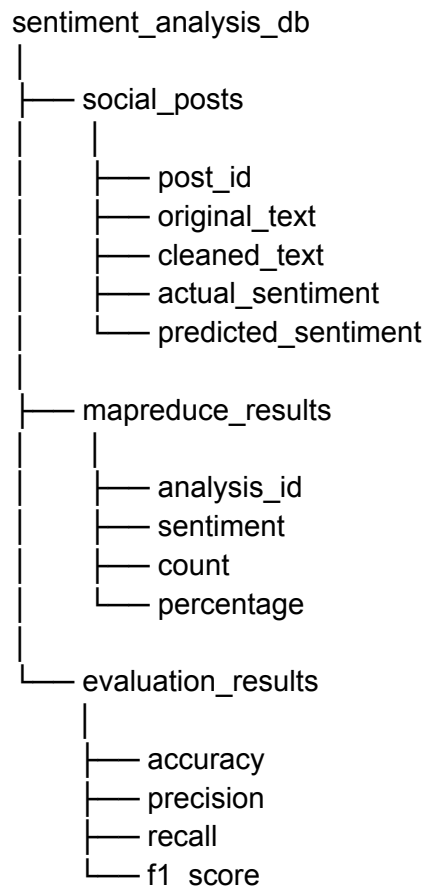
Hybrid Approach



↓
Store Aggregated Results

This is easier to implement and gives you full control over the Mapper and Reducer.

Suggested MongoDB Collections Relationship



Suggested Final Features

Your completed project should support:

Dataset Features

- Dataset fetching
- CSV generation
- Data preprocessing

Database Features

- MongoDB storage
- Batch insertion
- Querying social media posts

Sentiment Features

- Positive detection
- Negative detection
- Neutral detection
- Sentiment score

MapReduce Features

- Mapper
- Shuffle
- Reducer
- Aggregation

Analytics Features

- Sentiment distribution
- Percentage analysis
- Dataset statistics

Evaluation Features

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

UI Features

- Interactive dashboard
- Charts
- Data tables
- MapReduce stage visualization